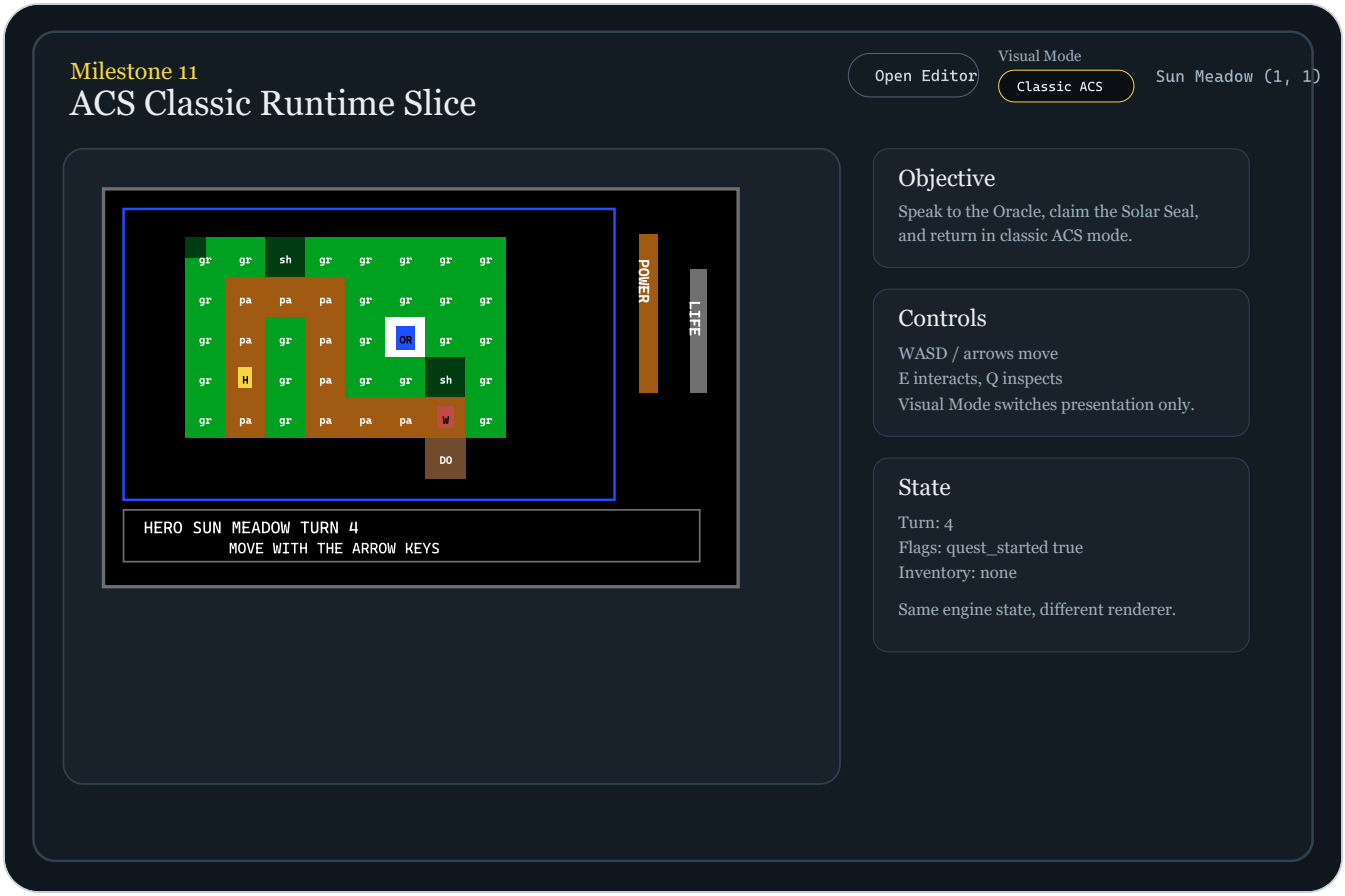
ACS MILESTONE 14

User Guide

This guide explains how to run the current application, use the classic ACS visual mode, play the sample adventure, save progress, use the editor, validate drafts, and move from a local draft to a published release.

What is included right now

- The playable runtime at `apps/web/index.html`
- The browser editor at `apps/web/editor.html`
- The local project, validation, and release API at `apps/api/dist/index.js`



Starting The Application

Build the workspace if needed, then start both servers from the repo root.

Web Server

```
node .\apps\web\server.mjs
```

Default URL:

```
http://localhost:4173/
```

Common alternate URL in this environment:

```
http://localhost:4317/
```

API Server

```
node .\apps\api\dist\index.js
```

Local API check:

```
http://localhost:4318/api/session
```

Editor URL

```
http://localhost:4317/apps/web/editor.html
```

Playing The Game

The runtime can load the built-in sample adventure, a local draft playtest, or a published release.

Sample objective

1. Speak to the Oracle.
2. Enter the shrine.
3. Claim the Solar Seal.
4. Return to the Oracle.

Keyboard controls

- `WASD` or arrow keys move
- `E` interacts
- `Q` inspects
- `Enter` or `Space` advances dialogue

Runtime panels

- Map and player position
- Session source
- Objective panel
- Save controls
- State summary and event log

Turn-based enemy timing

Successful player actions advance the turn count, but blocked movement does not give enemies a free action. Enemy behavior can define a `turnInterval`; the sample Shrine Wolf acts every third successful player turn, giving you room to maneuver instead of shadowing every step.

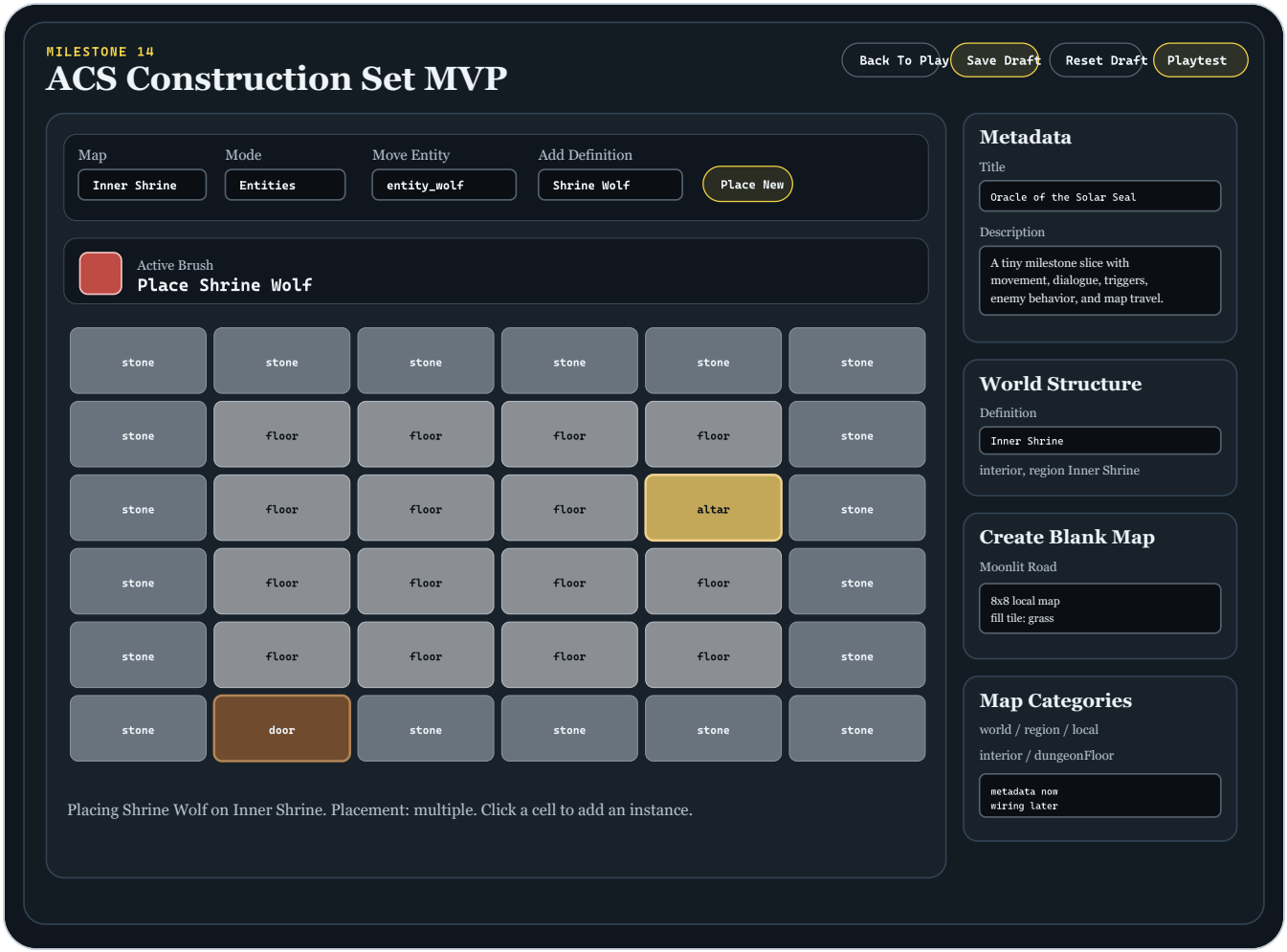
Saving And Loading Progress

The runtime uses separate local save slots for the sample adventure, draft playtests, and published releases.

- **Save** stores the current runtime snapshot in IndexedDB.
- **Load** restores the most recent snapshot for the current source.
- **Reset** restarts the live session without deleting the saved snapshot.

Using The Editor

Open the editor at `http://localhost:4317/apps/web/editor.html`.



The current editor supports metadata editing, tile painting with a persistent brush, adding and repositioning entity instances, reusable entity definition editing, dialogue text editing, structured trigger editing, map structure editing, blank map creation, shared validation summaries, API-side `Validate Draft` checks, local draft saves, backend project saves, publishing releases, and opening those releases in the runtime.

Top buttons

- `Back To Play`
- `Save Draft`
- `Reset Draft`
- `Playtest Draft`

Toolbar controls

- `Map` chooses the current map
- `Mode` switches between Tiles and Entities
- `Tile` picks the tile to paint
- `Move Entity` picks the existing entity to move
- `Add Definition` chooses the reusable entity definition to place
- `Place New` switches entity mode into new-instance placement
- `Active Brush` shows the tile brush or entity placement target

Tile Editing

1. Choose `Tiles` mode.
2. Select a tile id from the dropdown.
3. Check the `Active Brush` preview to confirm the selected tile.
4. Click a cell in the grid to paint a single tile, or click and drag to paint across multiple cells.

The selected tile stays loaded like a brush until you choose a different tile.

Entity Editing

Entity mode supports both moving existing instances and placing new instances from reusable definitions.

1. Choose `Entities` mode.
2. To move an instance, pick it from `Move Entity` and click a destination cell.
3. To add a new instance, choose a definition from `Add Definition`, click `Place New`, then click a destination cell.

Placement rules

- `singleton` definitions can appear once in the adventure. The Oracle is singleton.
- `multiple` definitions can be placed repeatedly. The Shrine Wolf is multiple.
- Validation reports a blocking error if a singleton definition has more than one instance.

Editing World Structure And Creating Maps

The `World Structure` panel edits the selected map name, category, and region assignment. The `Create Map` controls create a new blank map with a chosen category, optional region, width, height, and fill tile. Milestone 14 does not yet create exits automatically; map wiring remains future work.

Editing Dialogue And Triggers

The `Dialogue Text` panel edits existing dialogue records. The `Rule Trigger` panel edits existing trigger records, including type, optional map/x/y location, run-once behavior, conditions JSON, and actions JSON. These JSON areas are structured package data, not executable scripts; invalid JSON is rejected until corrected.

Validation

The validation panel runs the shared validation package and shows an error-and-warning summary plus a detailed issue list for the current draft. Use `Validate Draft` in the `Project & Release` panel when you want the local API to run the same validation report the publish flow uses. Blocking errors keep project save and publish actions disabled until they are fixed.

Tutorial: Make A Simple Map Edit And Play It

Because the current editor cannot create a brand-new map yet, the easiest useful tutorial is to modify one of the built-in maps and then playtest the edited version.

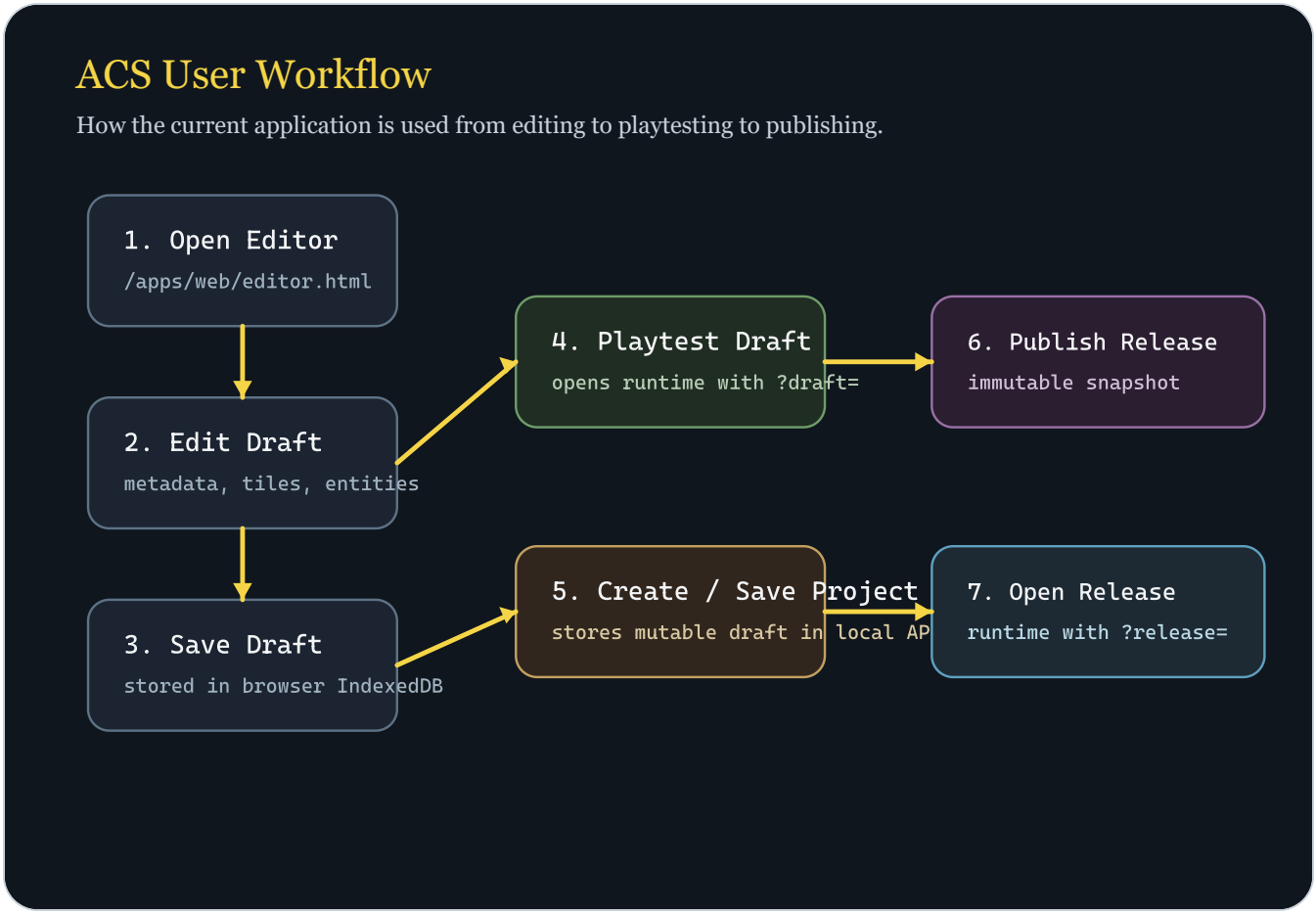
Goal of this tutorial

We will edit `Inner Shrine`, paint a few tiles to make the room feel different, validate the draft, save it, and open that draft in the runtime.

1. Open the editor at `http://localhost:4317/apps/web/editor.html`.
2. Choose `Inner Shrine` in the `Map` dropdown.
3. Set `Mode` to `Tiles`.
4. Select `floor` in the `Tile` dropdown.
5. Use the selected tile like a brush. Click individual cells or hold the mouse button down and drag across multiple cells to paint continuously.
6. Leave the `door` tile in place so the map exit still works.
7. Leave the `altar` tile in place if you still want the shrine reward trigger to work.
8. Click `Validate Draft` and confirm the draft is publishable.
9. Click `Save Draft`.
10. Click `Playtest Draft`.
11. In the new runtime tab, walk to the shrine and confirm that the room layout matches your edits.

If the local API is running, you can continue by clicking `Create Project`, `Save Project`, `Publish Release`, and then `Open Latest Release`.

Projects And Published Releases



The editor can move a draft through five project stages:

1. **Validate Draft**: run the backend validation report without publishing.
2. **Create Project**: create a mutable backend project from the current draft.
3. **Save Project**: update the mutable backend draft.
4. **Publish Release**: create an immutable release snapshot.
5. **Open Latest Release**: launch that published release in the runtime.

Important distinction

- `Save Draft` writes to browser storage.
- `Save Project` writes to the local API.
- `Publish Release` freezes a release snapshot rather than editing it in place.

Where Data Lives

Browser storage

- Runtime saves
- Local drafts
- Remembered active project id

Local API storage

- Mutable project drafts
- Immutable published releases
- Stored locally in `apps/api/data/store.json`

Current Limitations

- No real user accounts yet
- No cloud backend yet
- No asset upload flow yet
- No deletion of maps or automatic exit/portal wiring yet
- No deletion of entity instances in the editor yet
- No brand-new trigger/dialogue creation yet; Milestone 13 edits existing records only
- The runtime still uses simple colored tiles and abstract markers

Troubleshooting

The editor says the API is unavailable

```
node .\apps\api\dist\index.js
```

Validate Draft fails or publish stays disabled

- A trigger may reference a missing map, item, dialogue, or quest.
- An entity or start position may be outside a map's bounds.
- A singleton entity definition may have more than one placed instance.

- A tile layer may have the wrong tile count for its dimensions.

A published release will not open

- The API may not be running.
- The release id may not exist anymore.
- The local API store may have been cleared.

My draft changes are gone

- Use `Save Draft` for browser-local storage.
- Use `Save Project` for backend storage.

Playtest Draft opens the sample adventure instead

The runtime falls back to the built-in sample when it cannot find the draft key passed by the editor.